

2. MMU translates using page table
3. Gets physical address
4. Access RAM.

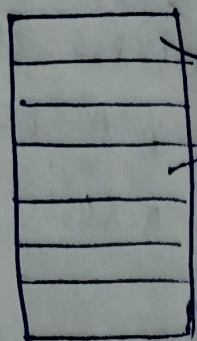
25/02

Paging:

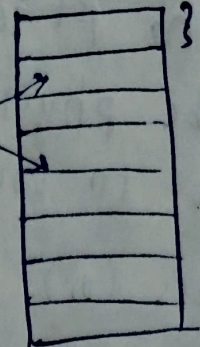
On compile time it gives

Pages of sample address $\rightarrow$ physical address (frames) page

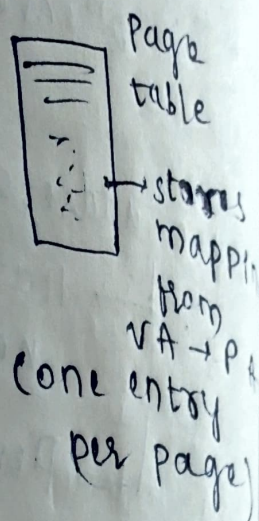
page



virtual address space



Physical address space



Drawbacks:

- $\rightarrow$ Access page table for every memory access (double no. of access)
- $\rightarrow$ Overcome (MMU) (overcome through MMU)

Two level page tables: (Motivation) $\Rightarrow$ $\boxed{\text{VPN} \rightarrow 20 \text{ bits}}$ $\uparrow$ $\boxed{\text{offset} \rightarrow 12 \text{ bits}}$

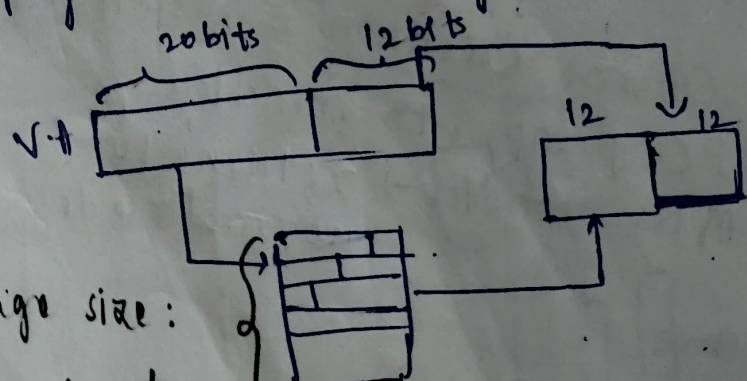
$\rightarrow$ Page table (VA $\rightarrow$ PA mappings) can occupy more than one frame

VA $\rightarrow 2^{32}$ Bytes
PA $\rightarrow 2^{24}$ Bytes

Page size = 2^{12} Bytes

$\Rightarrow$ size of each entry in page table (PPN $\rightarrow$ Physical Page No.) $\boxed{12\text{-bits}}$

do not fit in bytes (Not a 8 multiple) (2 bytes per entry)



page size:

No. of entries:

Page table

PTE $\rightarrow$ Page Table Entry

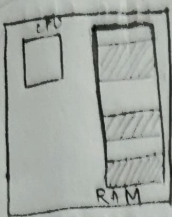
No. of PTE's = $\frac{2^{32}}{2^{12}} = 2^{20}$

Size of page table = $2 \times 2^{20} = 2 \text{ MB}$

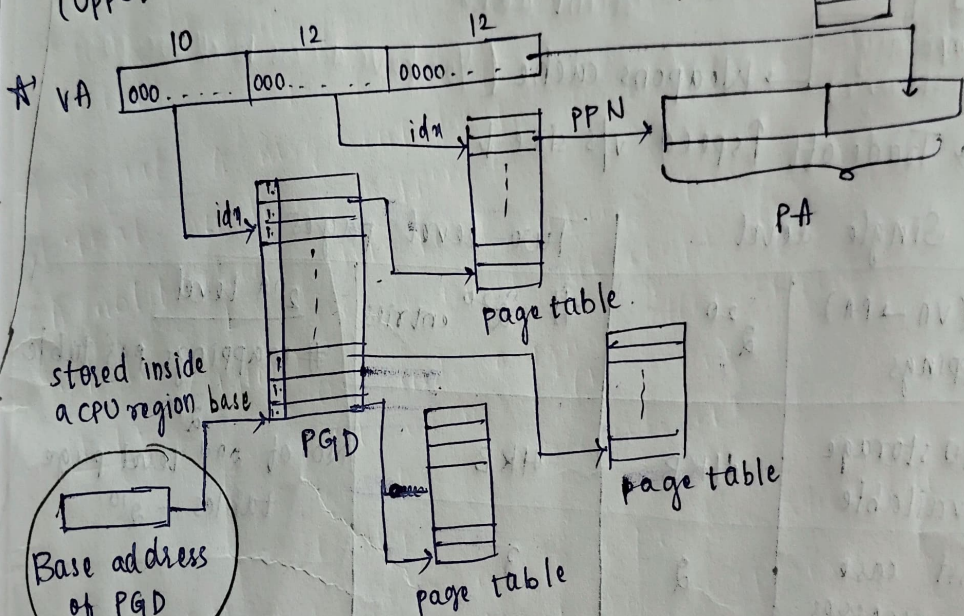
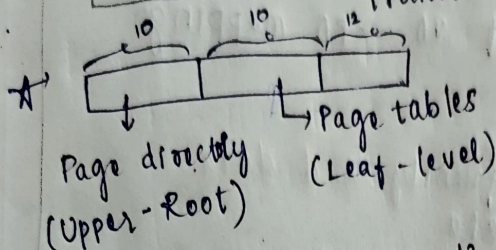
(But physical RAM is only 2^{24} B is the scenario)

No. of page frames = $\frac{2^{24}}{2^{12}} = 2^{12} = 4096 \text{ frames}$

page tables are process specific
1 table/process



Mappings are stored in RAM
→ Makes it inefficient/feasible to keep the entire page table inside the CPU (MMU) for faster address translation



★ MMU (inside the CPU) → VA
→ Base address of PGD

→ As processor generates the VA

Step 1: Look up PGD (1 memory access)

Step 2: Look up only the page table whose base address is given by step 1. (1 mem. access)

Step 3: Append the offset to get PA

Step 4: Issue memory access offset using PA (1 mem. access)

You reduce the amount of storage
→ CPU need not store everything

MMU → can store PGD & a frame page tables internally

2 level page tables → worst case scenario, we require 3 memory accesses per load/store issued by the processes

Assumption
 ⇒ Page directory and the current page table can be stored inside the CPU (after the 1st case) (Step-1 & Step-2)
 This saves 2 additional memory access into CPU

* 2 level page table → 3 mem. accesses (to get (Per level 1 access + 1 for PA))

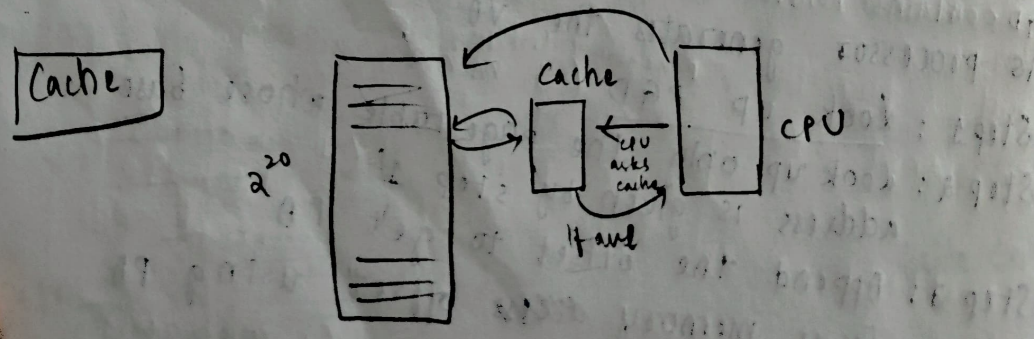
Scenario:

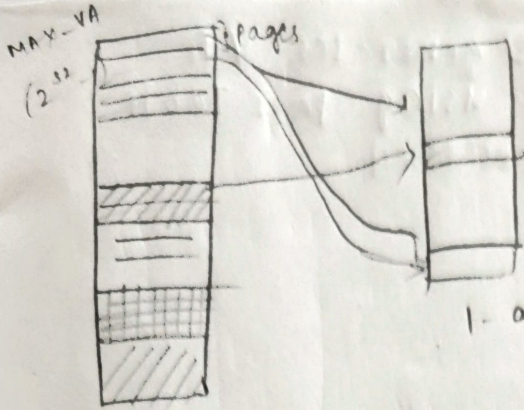
CPU/MMU → store HKB of VA → PA mappings (It takes any 4KB)

"Cache" → Stores most frequently used data recent data

(Temporary storage) → Weapons cache (You have access to all weapons)
 → Trade off ↑ speed v/s size ↓

Single level		Two-level pages	
# (VA → PA) mappings	2^{20}	# 2^{20} entries	2 nd level # mappings per table = 2^{10}
MMU storage available	4KB	4KB	No. of 2 nd level page tables = 2^{10}
Worst case mem accesses	2	3	





Some VA $\rightarrow$ PA mappings do not exist (Where is this captured in page table)

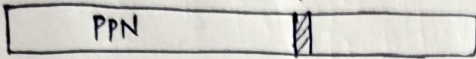
1- additional bit $\rightarrow$
Take any address, are pointing legal offsets ~~you~~

PTE:

valid

$2^{10} + 1$ bit

If some mapping is not mapped $\rightarrow$ PAGE FAULT



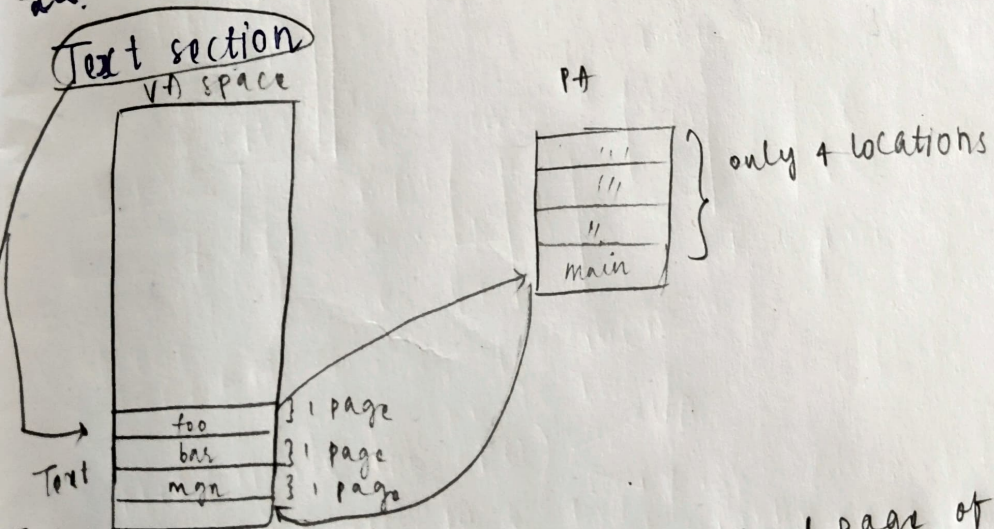
only if valid,

Step-1: Look up PGD and return Page Table Base, if not valid raise "error"

$\Rightarrow$ The MMU (H/W) raises an "error" if a VA $\rightarrow$ PA mapping is invalid (PAGE FAULT)

In response to a PAGE FAULT, the fault handler gets invoked. Fault handler is part of OS.

Ex: VA space $\gg$ PA space



Textbook:
Pagetable: To record where each virtual page of the address space is placed in physical memory, the O.S usually keeps a per-process data structure known as page table. It stores address translations

9/3/26

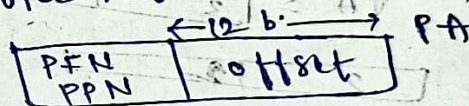
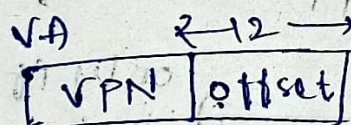
Address Translation

VA $\rightarrow$ PA

Mapping is set by the OS

Mappings are used for translating memory accesses (load/store)

(Address translation)

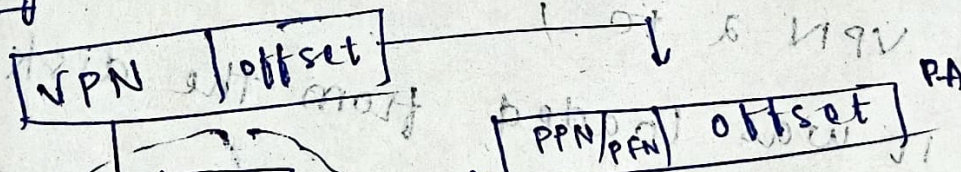


VPN $\rightarrow$ virtual page No.

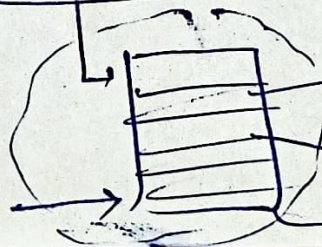
PFN - Physical Frame No.

PPN - Physical Page no.

Mapping:



$\therefore$ Page address



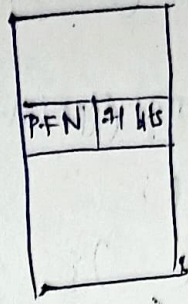
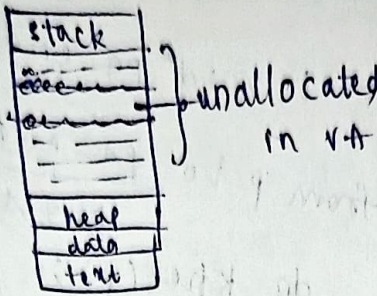
Page Table entry (PTE)

Page Table written by OS

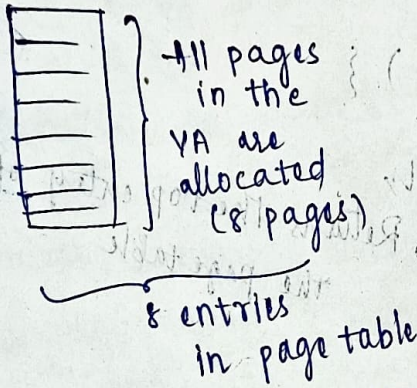
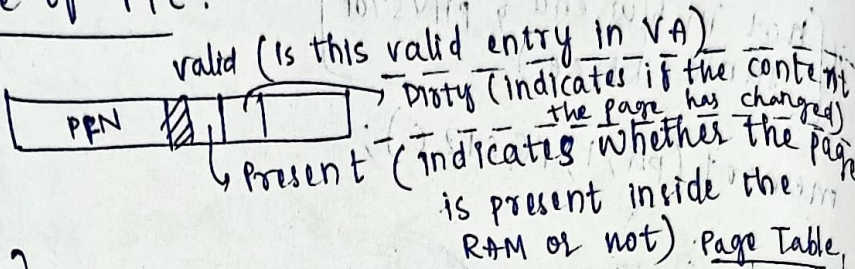
VA space is larger than PA space

$$(MAX - VA : 2^{32} - 1)$$

$$(MAX - PA = RAM \text{ size})$$



Structure of PTE:



RAM

VPN	VA
0	0
1	1
2	2
3	3
4	0
5	1
6	2
7	3

RAM

VPN	VA
0	0
1	1
2	2
3	3
4	0
5	1
6	2
7	3

empty

VA = VPN

VA = 6 | 2

→ This case is when you start the program
 → Now takes the data from disk.



(root/boot) VA = 5 | 1000

VA = 0 | 1000

VA = 3 | 1000

What happens now?

Now VA = 2 | 1000

Steps:

- 1) Write VPN 6 to disk
- 2) change the present bit agent VPN 6 to 0
- 3) load VPN-2 from disk
- 4) Set present VPN 2 to 1

Dirty → since it was loaded from the disk.

```

ex: for (i=0; i < N; i++) {
    a += arr[i];
}

```

arr[i] → Reads
a → Reads + writes

- load page(a) into RAM
- load 'a' into register (Not yet modified)
- ~~write 'a' into disk~~ Not necessary because a was not modified
- load arr into RAM
- load arr[i] into register
- addition of a & arr[i]
- load a into RAM
- write a into RAM → Dirty = 1 (a is changed)
- load a into register → write a into disk Because dirty = 1
- load 'arr' into RAM

★ Which page to replace?

Free list → list of page frames that are free
if (free-list is null) & all page frames are occupied
find a page to replace:

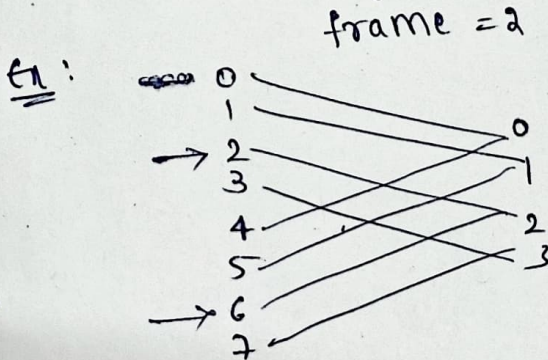
VA

2	3
---	---

tweak

↳ Do you have the freedom to ~~create~~ tweak the mappings.

→ Under a given mapping, how do you identify which page to replace.



$4(0) + 2$
 $4(1) + 2$

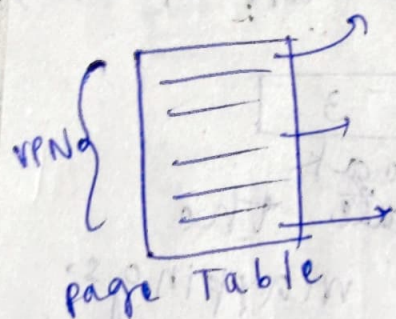
same
RAM
pointer
no.

Assume: All accesses to
VAN 6 is reads.

11/03

→ Multi-level Paging:

VA → PA ⇒ Mappings can be just 1 large table

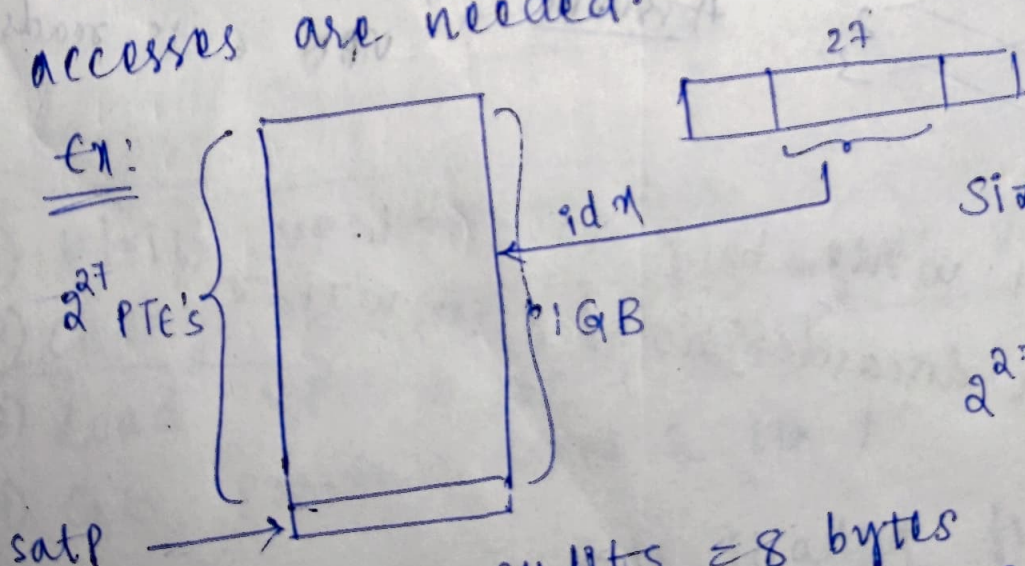


Drawbacks of multi-level paging

→ (# levels of 1) ⇒ No. of mem accesses necessary to perform 1 VA access

(The picture from text book is explained)

The PPN of L2 tells the base address of page directory of L1 and the same way L1 → L0. At last the PPN is then mapped to physical address PPN. } Page Table walk
 → MMU storage which avoided multiple accesses could run out of space in which case (# levels + 1) accesses are needed.



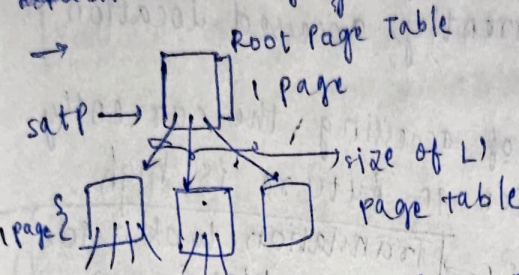
Size of 1 PTE = 64 bits

$$2^{27} \times 2^6 = \frac{2^{33}}{2^3} \text{ bits}$$

Size of 1 PTE = 64 bits = 8 bytes
 1 page table (at any level) = 2^9 PTEs = 2^{30} bytes

Size of 1 page table (at any level) = $2^9 \times 2^3$
 $= 2^{12} = 4KB$

Multi-level Paging Page size



Benifits of multi-level paging:

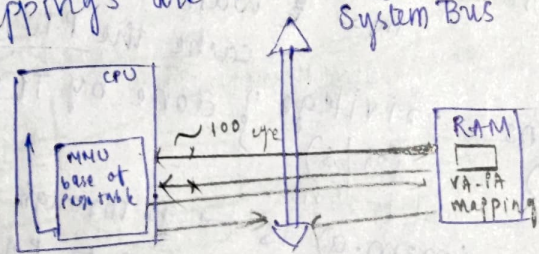
→ Page table need not be contiguous in the memory
 (page the page table itself)

Structure of PTE

Valid
 "Present" → Demand Paging (Pages are brought into frames on-demand)

If we could find an address in main memory then we have to remove 1 entry of page RAM & load the required one from disc
 [present → 0]

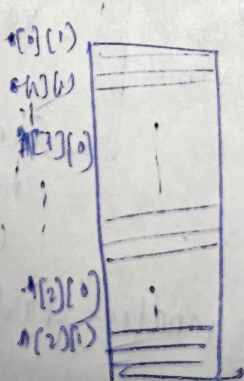
→ Need not create intermediate page tables if such mappings are invalid (beneficial for sparse mappings)



Cache: Trade off between Storage capacity v/s Speed of access

* Cache of VA-PA mappings

ex:
 for (i=0 - n)
 for (j=0 - n)
 arr[i][j] = 0



VA:
 (base),
 (base+4),
 (base+8),
 ;

Locality of references:

spatial

→ The likelihood of accessing memory locations adjacent to the currently accessed location is high

temporal → The likelihood of accessing the currently accessed location in future is high

Cache of VA-PA mappings:

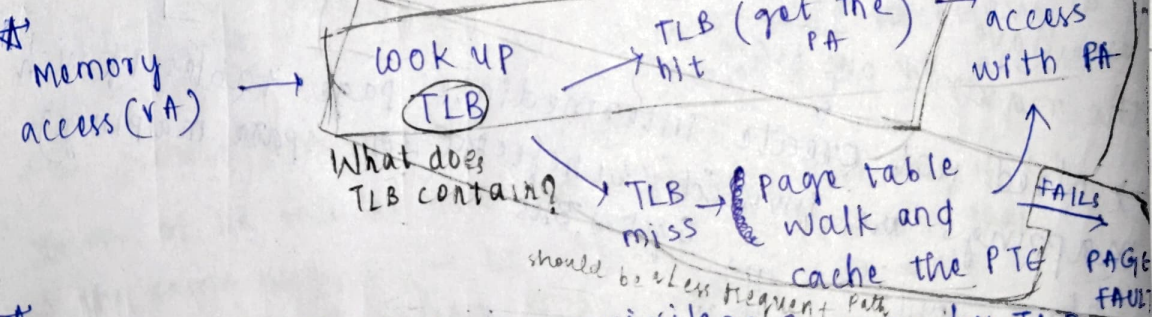
Translation look-aside buffer (TLB)

PTE's

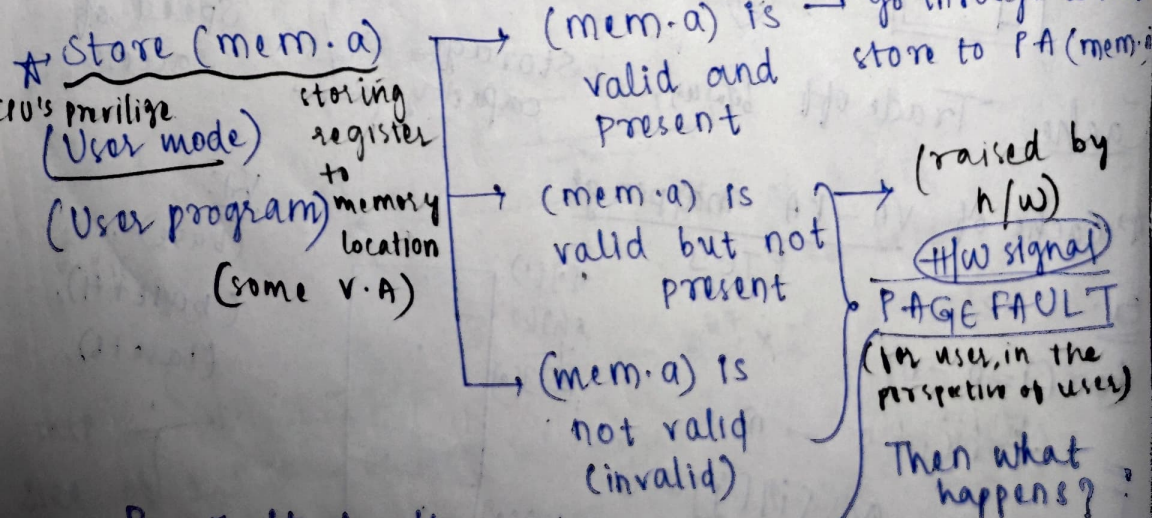
→ cache (or) store the recently used PTE's inside the MMU

* CPU generates memory access VA → look up TLB

TLB → hit → VA-PA mapping is present in cache
 TLB → miss → VA-PA mapping is not present in cache. more frequent path.



(PTE's) ⇒ Permissions (R W X), privilege (u/s) } done by TLB
 user/supervisor



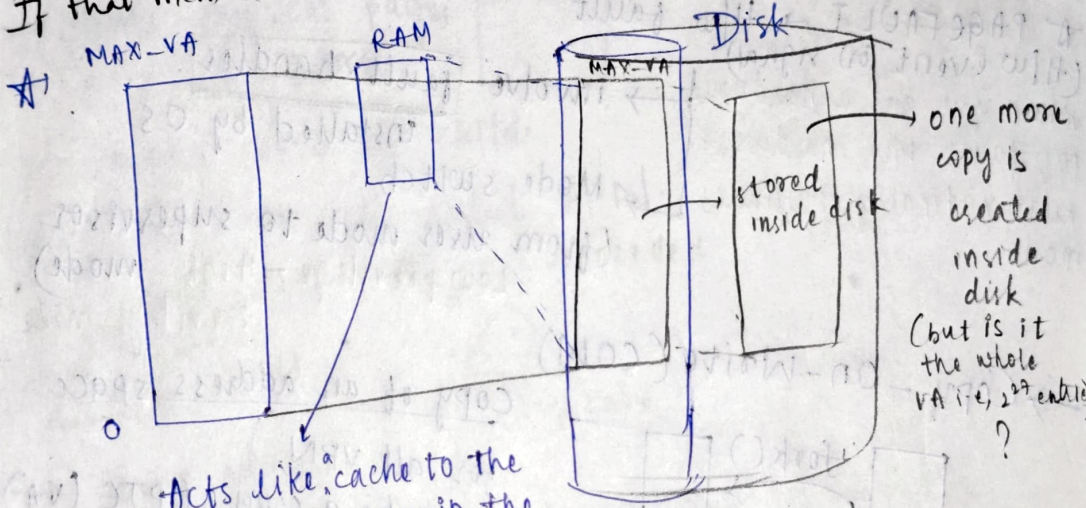
PageFault handler starts executing

checks if valid, is 1 swap the page into the RAM & returns to faulting past

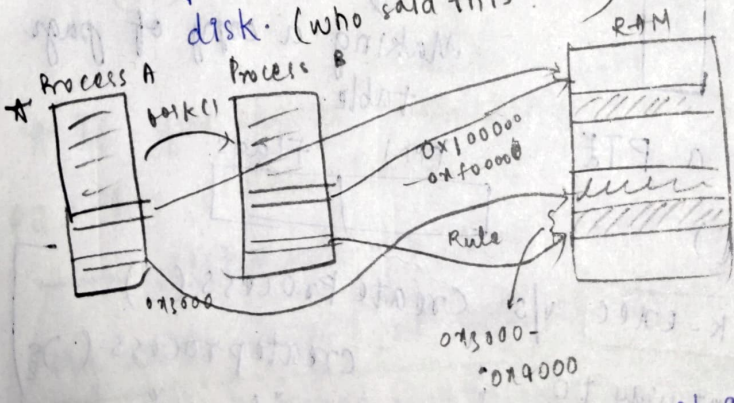
valid = 0 → Terminates the program
 → generate SIGSEGV
 → Generates SIGKILL / SIGTERM

→ OS just sets the mapping given by MMU (OS doesn't keep coming every now & then, OS coming & doing a work is very expensive thing)
 So you give some work to h/w.
 (Trap v/s Fault)

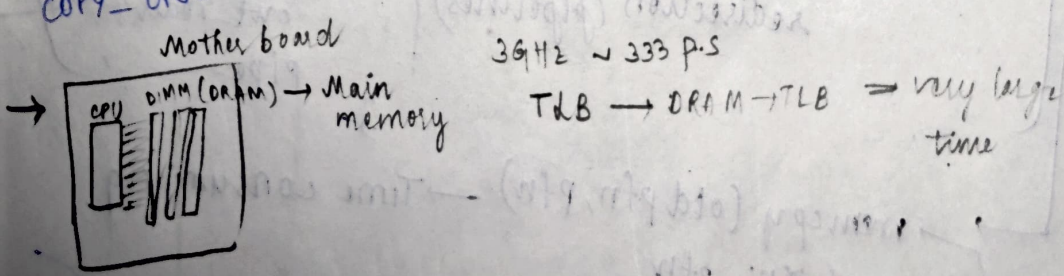
→ If that mem. a was div by zero → not fault → it's trap

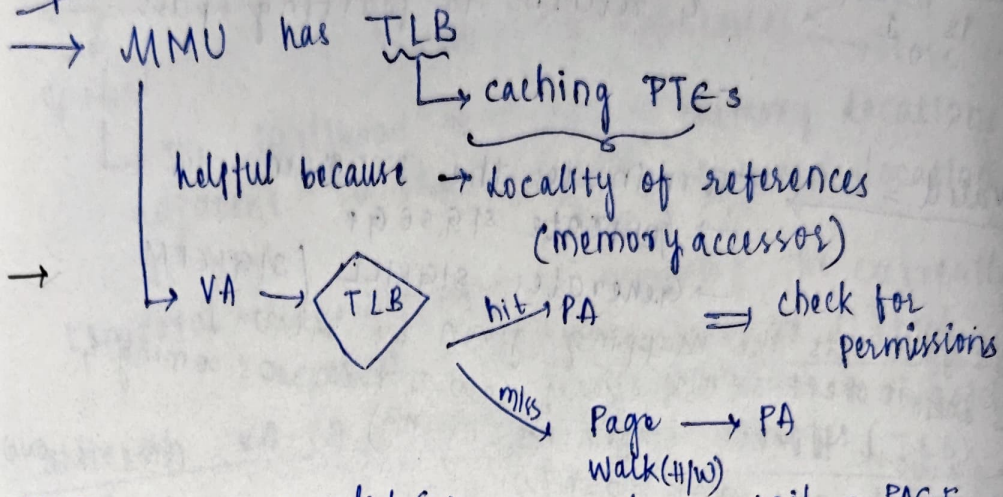


Acts like a cache to the process image in the disk. (who said this? → OS)



THIS IS COPY-ON-WRITE → make a copy of a page only when you want to write to it.



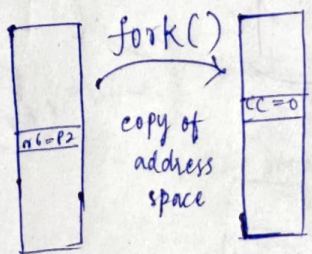


* If page is invalid (or) perm checks fail → PAGE FAULT

* PAGEFAULT → H/W fault (H/W event (or) signal)
 when you get a hardware pagefault the hardware automatically switches mode

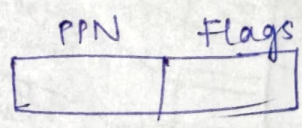
- involve fault handler installed by OS
- Mode switch (from user mode to supervisor mode)
 Low privilege → High mode

→ Copy-On-Write (COW)



Copy of an address space for all VPN,
 make a copy of PTE (VA)
 ↓
 Making a copy of page table.

→ Make a copy of a PTE
 PFN = kalloc()



* Benefit of fork-exec v/s CreateProcess()

elegant way to implement I/O redirection (pipelines)

```

createprocess ( "cmd.exe",
    "/a.out",
    in_redir,
    out_redir,
    pipe,
    ...
    )
    
```

memcpy (old pfn, pfn) → Time consuming
 return pfn

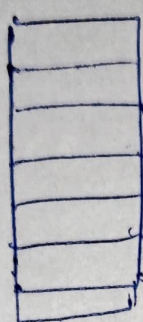
exec()

COW:

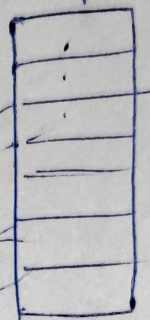
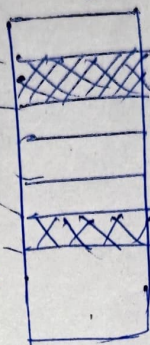
→ Copy the contents of a shared (physical) page only if it changes in one of the address space

ex: P1 (VA) - fork

P2 (VA)



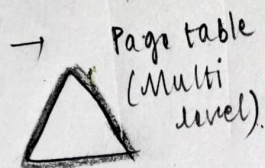
Physical address space



⇒ Share physical pages b/w the parent and the child.

(We want both the page tables to store the same)

⇒ Go over all valid VPN.



Page table (Multi level)

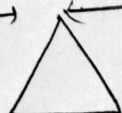
copy

Not needed



Page table

→ Base address (P2)



base address of level 2 (root level) (P1)

* If you change something in P1, it changes in P2. This violates isolation

→ Although efficient, but it breaks "Isolation". (My write should be ^{not} readable / "seen by anyone")

*** [reads are shared → writes are private]

→ There will be some permissions of r/w/x

PTE (P1)

PTE (P2)

